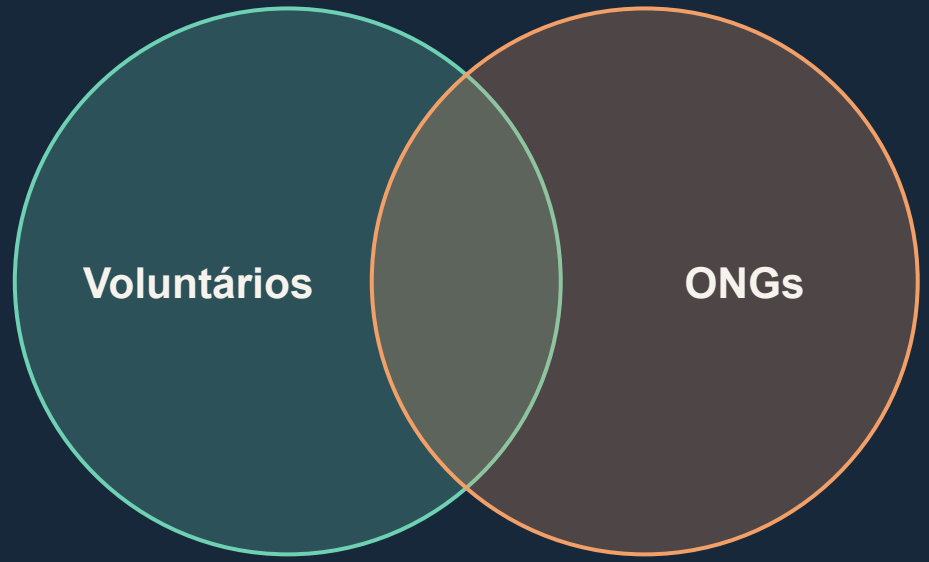
MDS 2026.2 · Documento de arquitetura

VolunJá

Decisões técnicas, implementação e testes



Equipe Gosling · MDS 2026.2

Monólito modular em camadas

Produto

Vagas e candidaturas com comprovantes, documentos, triagem e contato.

Escolha

SPA React e API FastAPI. Regras no servidor e módulos com dados no MySQL.

Por que esse estilo

Quatro pessoas, seis sprints. Python e React já conhecidos. Um servidor, módulos testáveis.

Alternativas avaliadas

MVC acopla a interface ao servidor. Microsserviços ampliam a operação. BaaS não atende à pilha Python definida pela equipe.

A separação em camadas permite desenvolver a interface e a API com um contrato comum.

Arquitetura e responsabilidades

Apresentação

React organiza páginas e componentes.
Axios concentra o acesso à API.

API e negócio

Routers e Pydantic recebem a requisição.
Services aplicam regras e transações.

Persistência e dados

Repositories usam SQLAlchemy.
MySQL e volumes persistem os registros.

HTTP/JSON e JWT, SQL, arquivos e SMTP conectam as partes da aplicação.

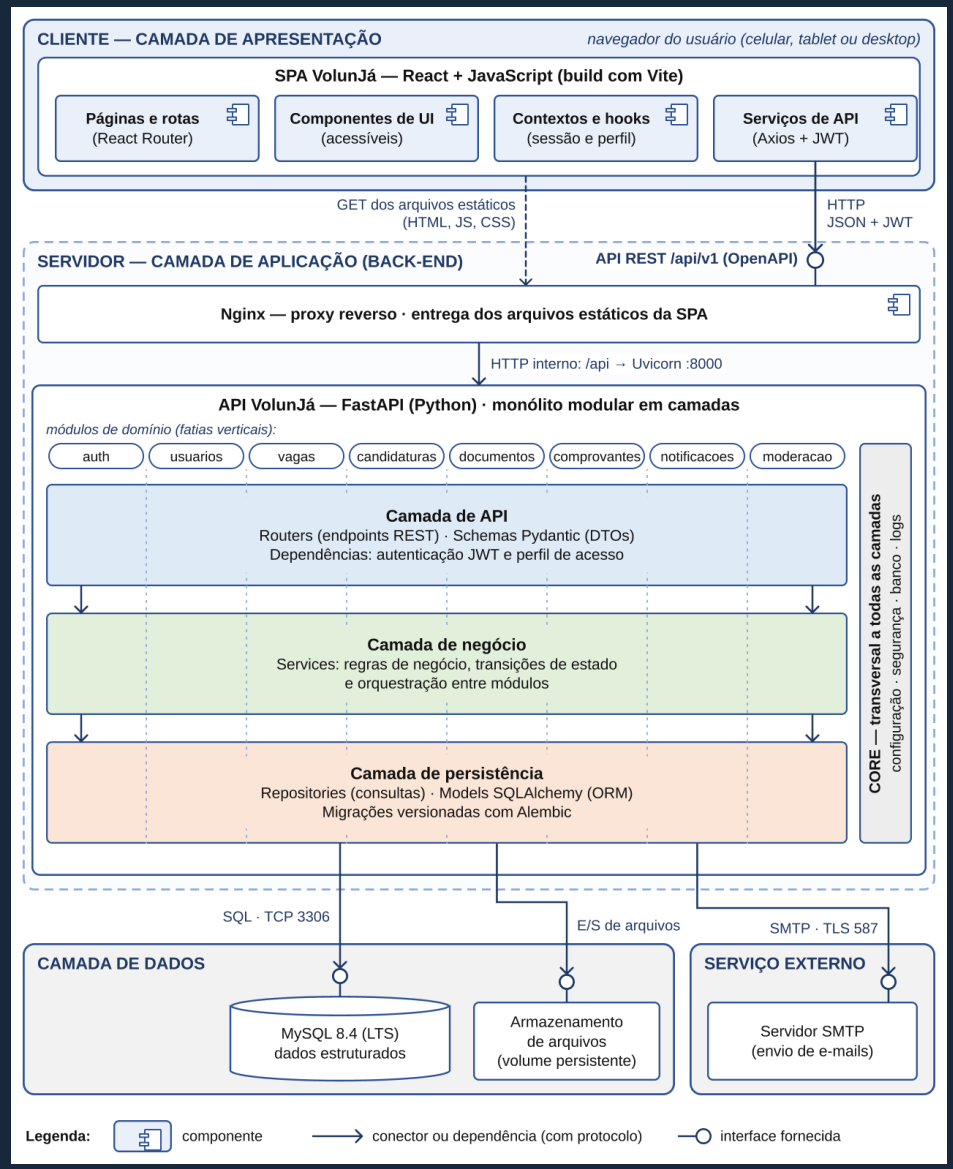


Figura 1 · Arquitetura do VolunJá

Escolhas técnicas e aplicação

Tecnologia	Por que escolher	Como vamos aplicar
React 19 + JavaScript	Componentes reutilizáveis e consistência de UX.	Telas por perfil, busca e formulários acessíveis.
Vite, Router e Axios	Build rápido, rotas organizadas e HTTP centralizado.	Build da SPA, navegação e serviços de acesso à API.
FastAPI + Pydantic	Validação de dados e contrato OpenAPI automático.	Endpoints, schemas e dependências de autenticação.
SQLAlchemy + Alembic	ORM e evolução versionada do esquema do banco.	Models, repositories e migrações no Git.
MySQL 8.4 LTS	Integridade referencial para entidades relacionadas.	Chaves estrangeiras e restrições de unicidade.

A pilha atende às tecnologias definidas no DV e à separação de responsabilidades do DA.

Perfis e funcionalidades do sistema

Visitante

Busca e filtra vagas.
Consulta detalhes e realiza o cadastro.

Voluntário

Mantém perfil, envia candidaturas
e documentos, recebe comprovantes.

Organização

Publica vagas, define documentos
e faz a triagem das candidaturas.

Administrador

Homologa organizações e modera
vagas, usuários e denúncias.

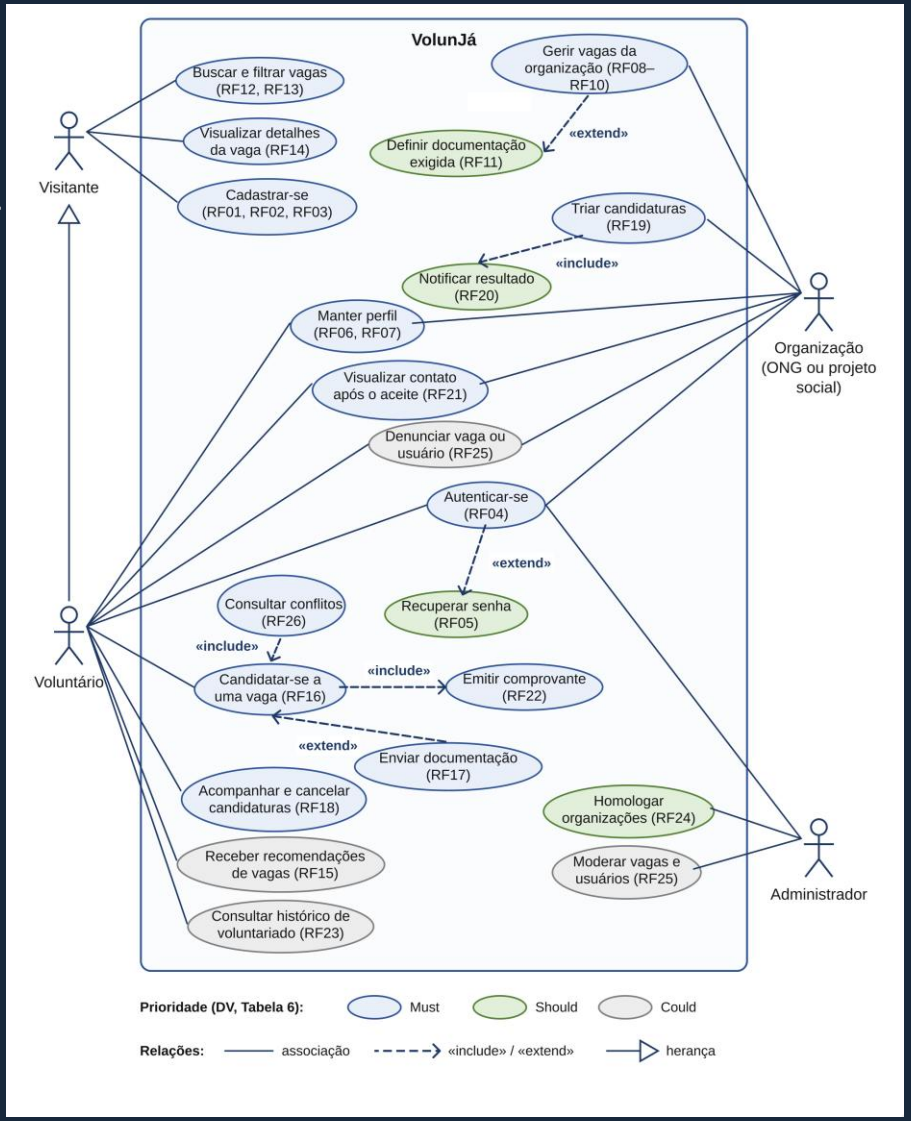


Figura 3 · Casos de uso e prioridades MoSCoW

Organização do código em pacotes

Front-end

Pages, components, contexts e hooks.
Services com Axios acessam a API.

Módulos de domínio

Auth, usuarios, vagas e candidaturas.
Documentos, comprovantes,
notificacoes e moderacao.

Estrutura padrão

router.py e schemas.py
service.py, repository.py e models.py

Core reúne segurança, configuração,
sessão de banco e tratamento de erros.

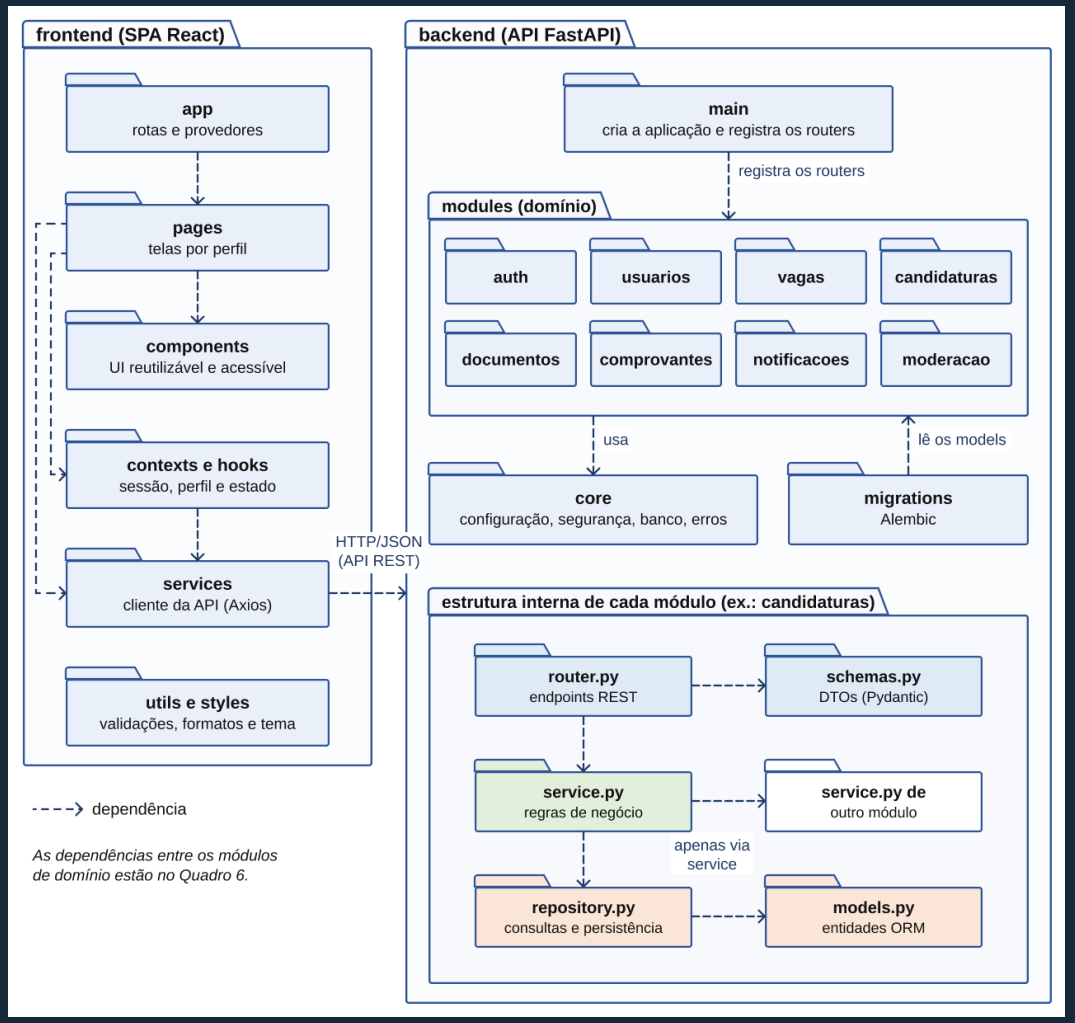


Figura 5 · Pacotes do front-end e do back-end

Componentes e integrações

Contratos entre subsistemas

A SPA consome a API REST /api/v1.
Módulos colaboram por services.

Dados e documentos

MySQL guarda os dados estruturados.
O módulo documentos controla
o acesso aos arquivos no volume.

Comprovantes e avisos

ReportLab gera comprovantes em PDF.
SMTP com BackgroundTasks envia
e-mails sem atrasar a resposta.

Dependências permitem testes isolados.

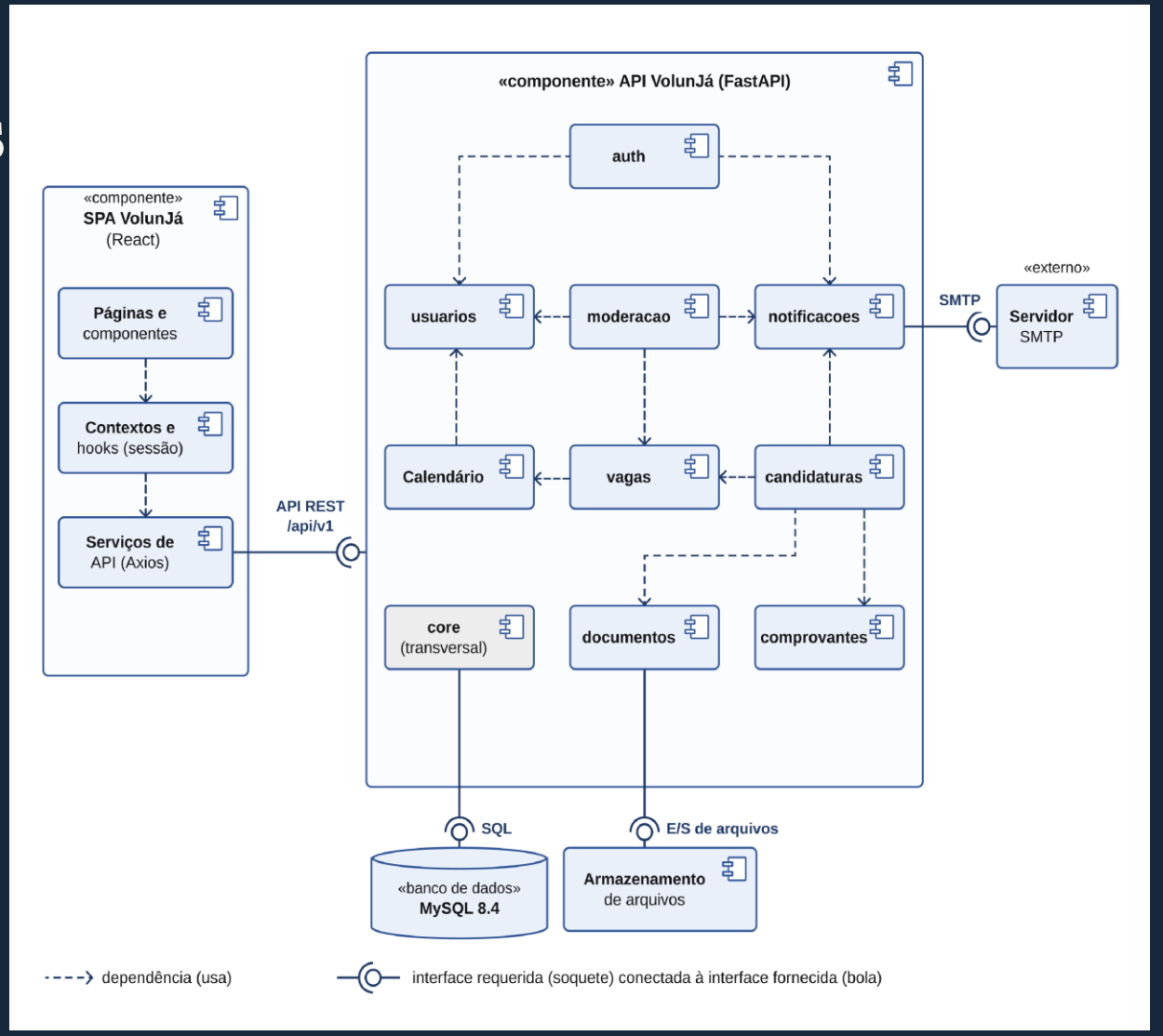


Figura 6 · Componentes e interfaces da aplicação

Entidades, relacionamentos e persistência

Identidade

Usuario se especializa em Voluntario e Organizacao. Cada conta tem um perfil.

Operação

Organizacao publica Vaga. Candidatura liga o voluntário à vaga. Causa e Disponibilidade apoiam a compatibilidade.

Documentos e registros

DocumentoExigido, DocumentoCandidatura e Comprovante. Notificacao e Denuncia registram avisos e moderação.

SQLAlchemy mapeia as tabelas do MySQL. Alembic versiona a evolução do esquema.

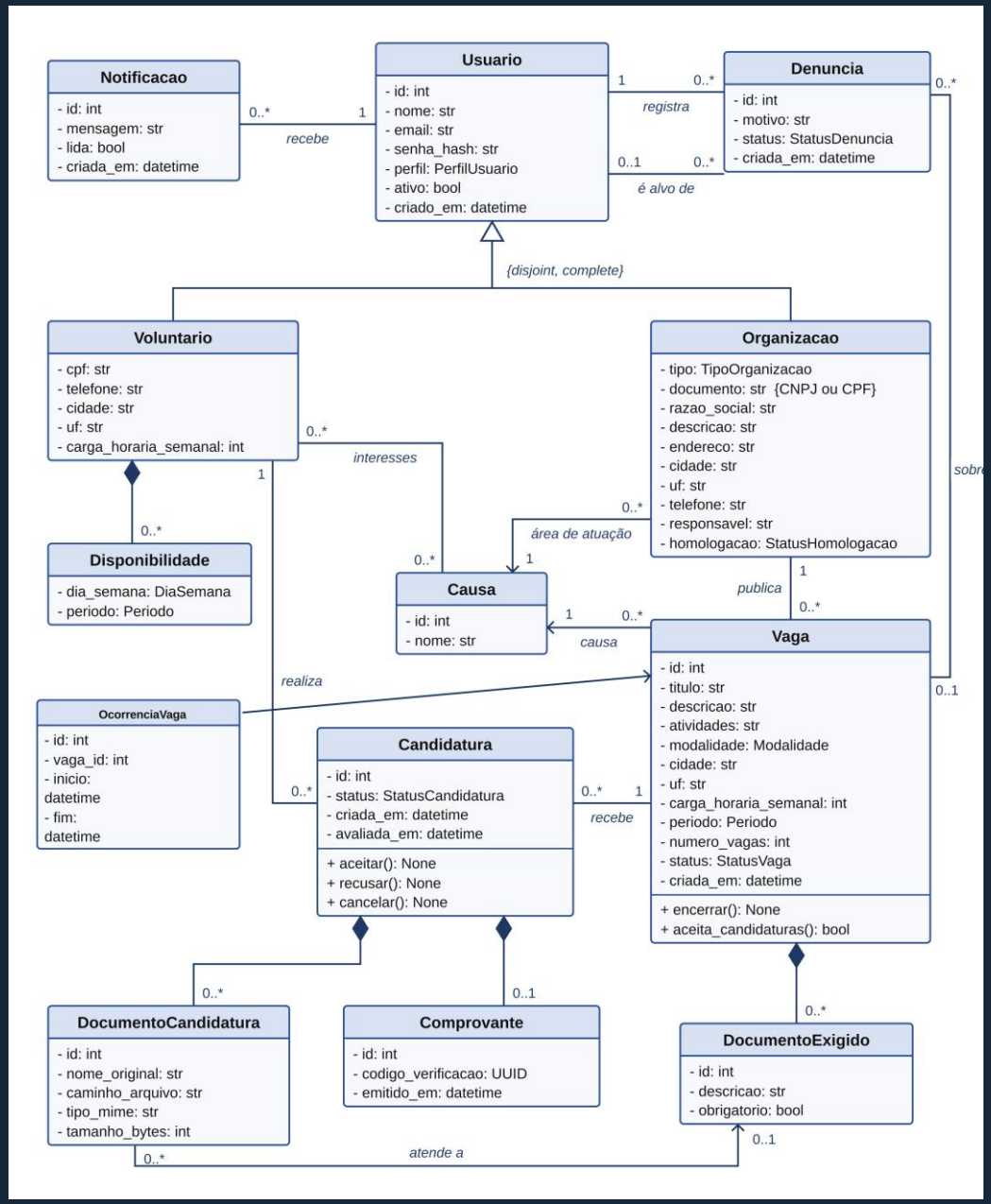


Figura 7 · Classes do domínio do VolunJá

Execução de uma candidatura

1 Contrato HTTP

O router valida JWT, perfil e entrada antes de chamar o serviço.

2 Regras e orquestração

O service verifica as regras de negócio e coordena os outros módulos.

3 Persistência

O repository persiste no MySQL.
O service controla a transação.

4 Resposta e notificação

HTTP 201 confirma o registro.
O e-mail segue em segundo plano.

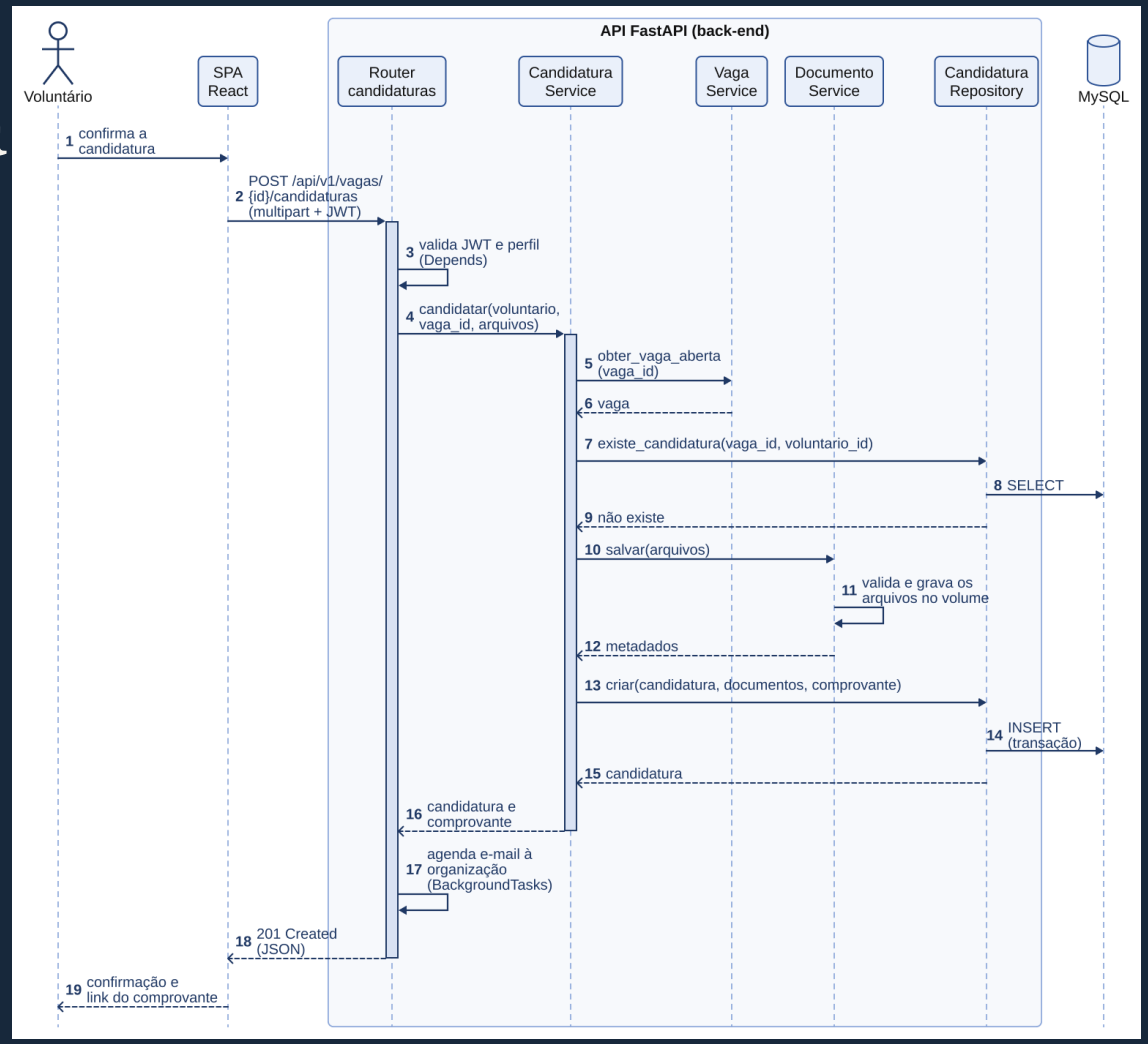


Figura 2 · Sequência de uma candidatura

Da candidatura à triagem e ao contato

Entrada

Busca, detalhes da vaga e autenticação preparam a candidatura do voluntário.

Validação

Vaga aberta, sem duplicidade e com documentos e horários válidos.

Registro

Candidatura pendente, documentos, comprovante e aviso à organização.

Triagem e conexão

A organização aceita ou recusa.
O aceite libera o contato entre as partes.

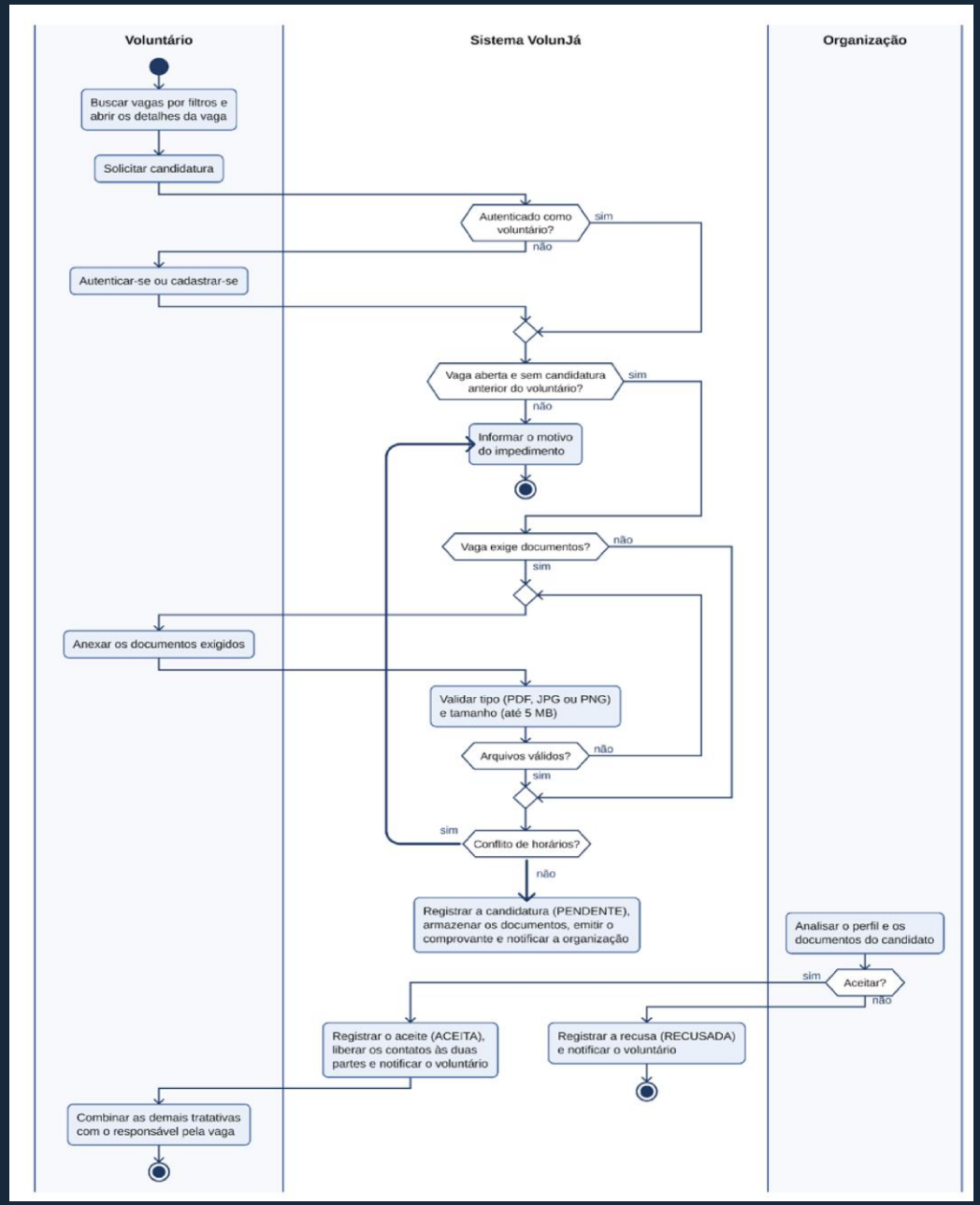


Figura 4 · Atividades de voluntário, sistema e organização

Aplicação da arquitetura no desenvolvimento

1 Contratos e dados

OpenAPI e Pydantic definem entradas e saídas.
SQLAlchemy e migrações Alembic estruturam o MySQL.

2 Funcionalidades por domínio

Services concentram as regras. React usa hooks, Axios
e CSS Modules com os padrões do protótipo no Figma.

3 Qualidade em cada pull request

Ruff, ESLint e Prettier padronizam o código.
GitHub Actions executa lint e testes com relatório de cobertura.

4 Revisão e entrega do incremento

Branch de trabalho entra na developer após revisão de um colega.
A main recebe o incremento validado ao fim da sprint.

Segurança e controle de acesso

JWT e Argon2

Token com expiração e senha protegida por hash. Autorização por perfil e por dono do recurso.

Validação e arquivos privados

Pydantic valida entradas. PDF, JPG ou PNG até 5 MB, entregues apenas pela API.

Perfil	Permissão central
Visitante	Consulta vagas e detalhes públicos.
Voluntário	Mantém perfil, candidaturas, agenda e documentos próprios.
Organização	Gerencia vagas próprias e as candidaturas recebidas.
Administrador	Homologa e modera. Não acessa documentos de candidaturas.

Testes e demonstrações usam dados fictícios. O MVP prevê até 100 usuários autenticados simultâneos.

Implantação do MVP e operação

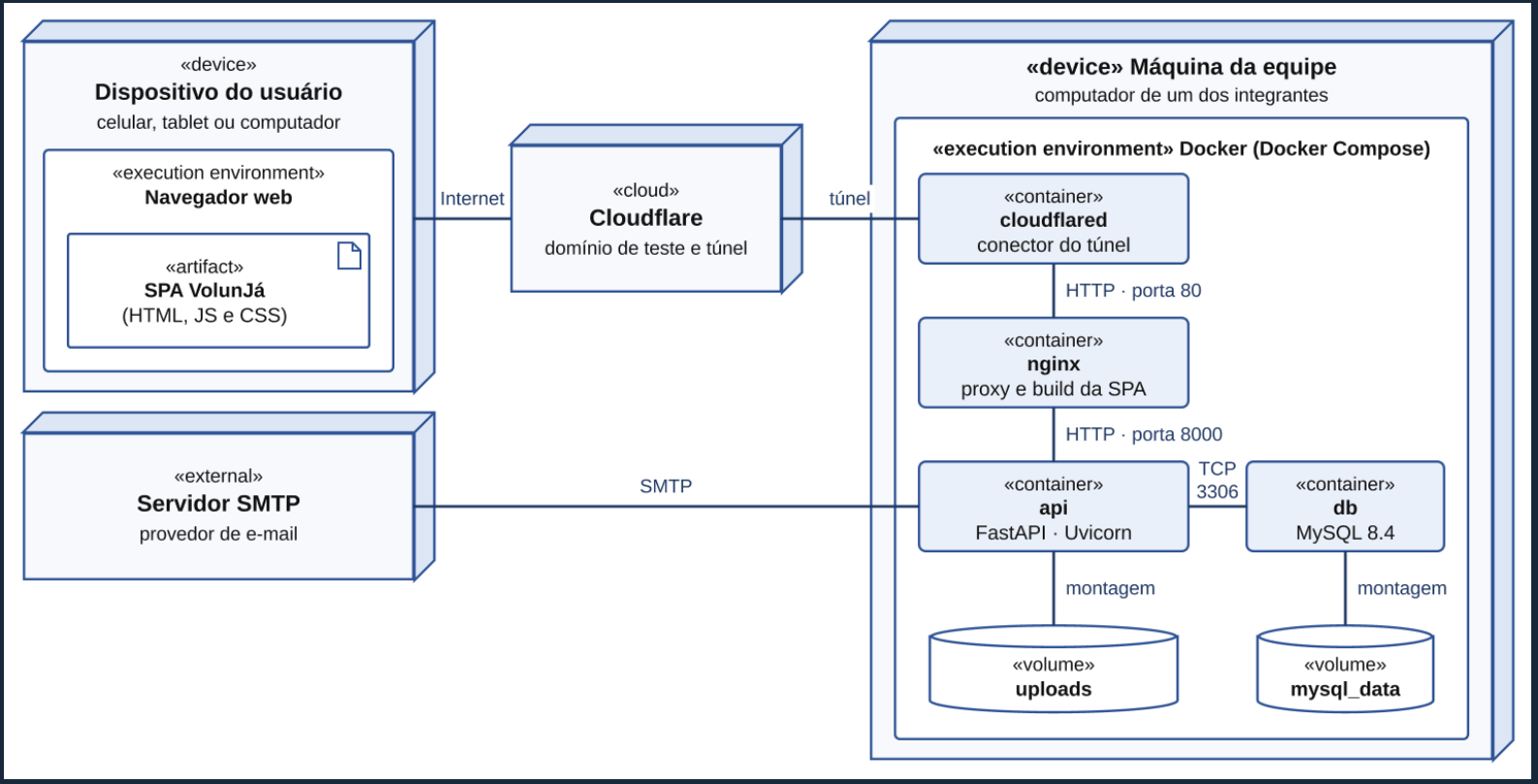


Figura 8 · Ambiente de implantação do VolunJá

Docker Compose

Mesmo ambiente para todos os integrantes.

Nginx e Uvicorn

Build da SPA, proxy /api e execução do FastAPI.

Cloudflare Tunnel

Publica o MVP da equipe em domínio de teste.

Volumes persistentes

Dados e uploads persistem. MySQL na rede interna.

Estratégia de testes e metas

Nível	Ferramentas	O que testaremos
Unitário	pytest Vitest + Testing Library	Regras dos services, validações e comportamento de componentes.
Integração	pytest e MySQL	API, repositories, permissões, transações e operações simultâneas.
Sistema	Playwright + axe-core	Cadastro, vagas, candidatura e triagem. Responsividade desde 360 px e acessibilidade.

CT01–CT14 cobrem o produto, incluindo documentos, controle de acesso e agenda.

≥ 80%

cobertura do back-end

≤ 2 s

95% das buscas, com 10 mil vagas

≥ 80%

sucesso em busca e candidatura